TVL for Tuned Systems

A Playful Engineering Companion

TraiGent Labs

October 16, 2025

Tuned Variable Language (TVL) is the specification backbone for governed adaptation in AI pipelines. This book introduces the motivation, constructs, and integration pathways for TVL, guiding software engineers and graduate students through real-world examples and project-ready patterns.

Contents

1 Why TVL Exists **1**
1.1 AI Pipelines Refuse to Sit Still . . . 1
1.2 Design Goals . . . 1
1.3 From Hard-Coding to Tuned Variables . . . 1
1.4 Running Example: Campus Orientation RAG . . . 2
1.5 How to Use This Book . . . 4
1.6 Field Notes Icons . . . 4

2 Getting Fluent in TVL **5**
2.1 First Contact . . . 5
2.2 A Minimal Working Spec . . . 5
2.3 Line-by-Line Walkthrough . . . 6
2.4 Typing Your Variables . . . 7
2.5 Validation Warm-Up . . . 7
2.6 Common Pitfalls . . . 8
2.7 Objectives and Budgets . . . 9
2.8 Promotion as a First-Class Citizen . . . 9

3 Constraints, Units, and Safety Nets **10**
3.1 Why Units Matter . . . 10
3.2 CEL in Plain English . . . 10
3.3 Worked Example . . . 10
3.4 Constraint Playbook . . . 12
3.5 Making Constraints Testable . . . 12
3.6 Debugging Failures Fast . . . 13

4 Patterns for Real Deployments **15**
4.1 Inheritance Without Surprises . . . 15
4.2 Staging vs. Production, Codified . . . 15
4.3 Constraint-Aware Overlays . . . 17
4.4 When to Extend vs. Duplicate . . . 17
4.5 Optimization Playbooks . . . 17
4.6 Hotfix Overlays . . . 18
4.7 Provenance and Promotion . . . 18

5 TVL Meets Triagent and DVL **19**
5.1 Unifying the Stack 19
5.2 Deterministic Artifacts 19
5.3 Handing Off to Triagent 20
5.4 Lifecycle of a Promotion 20
5.5 DVL Keeps the Data Honest 20
5.6 A Manifest That Glues It Together 20
5.7 Automation Recipe 21
5.8 TVL Tooling and Plug-ins 22
5.9 Looking Ahead 22

A Quick Reference and Resources **23**
A.1 Compiling This Book 23
A.2 Essential Commands 23
A.3 Further Reading 23
A.4 Exploring the Examples 23
A.5 Toolchain Map 24

Chapter 1

Why TVL Exists

1.1 LLM Pipelines Refuse to Sit Still

Large language model development today is notoriously under-specified. Prompts, retrieval depth, model choice, and safety filters often live in ad-hoc notebooks or scattered YAML fragments. When a researcher hands off a "working" configuration, the implicit assumptions rarely make it into production. The SANER extended abstract reminds us that the world around our language model systems is volatile: model catalogs shift, latencies spike with traffic bursts, and price lists change without warning. TraiGent's Tuned Variable Optimizer (TVO) frames this volatility as an engineering problem—the optimal configuration is not a single hard-coded value, but a moving target $v^*(E, I)$ conditioned on environment E and workload I. Traditional configuration files cannot keep pace, so engineers either over-provision (wasting budget) or accept drift (sacrificing quality). TVL is born from that friction.

1.2 Design Goals

TVL is a specification language that captures the semantics TVO needs to search safely:

- **Type-safe tuning variables** that distinguish categorical choices from continuous ranges.
- **Deterministic exports** so configuration can be generated, versioned, and reproducible.
- **Constraint grammar** (via CEL) to encode business and safety rules alongside the numbers.
- **Governed evolution** with promotion gates, reviewers, and audit trails.

Instead of scattering this knowledge across spreadsheets, dashboards, and ad-hoc scripts, TVL centralises it so pipelines can adapt while staying well-behaved.

1.3 From Hard-Coding to Tuned Variables

Imagine maintaining an onboarding chatbot the night before the semester starts. A new model checkpoint drops, GPU prices wobble, and policy reviewers add a fresh refusal requirement. With hand-tuned prompt files you either ship blind or pull an all-nighter. TVL gives you a different

move: express the feasible configuration space, tell TVO the goals, and let the optimizer search within the guardrails. When conditions shift again, you rerun the same playbook without rewriting application code. This reframing—LLM configuration as a governed, iterative artifact—is the core mindset change we want you to carry through the rest of the book.

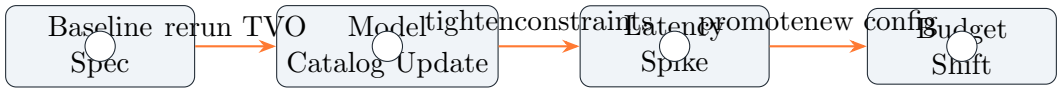

Figure 1.1: TVL encourages iteration: each environment change kicks off a repeatable optimization loop instead of a frantic patch.

1.4 Running Example: Campus Orientation RAG

Throughout the book we will revisit a retrieval-augmented generation (RAG) assistant that answers questions from incoming students. The workload is quirky: traffic spikes during orientation week, budgets tighten after fiscal close, and latency promises must hold even when queries flood in at 8am. The TVL snippet below is our starting point.

Listing 1.1: TVL spec for the campus orientation RAG assistant

```
spec:
  id: rag-campus-orientation
  version: 0.1.0
metadata:
  owner: "mlops@traigent.ai"
  description: >
    Adaptive RAG configuration for onboarding questions that drift with
    semester timelines, budgets, and latency promises.
  tags:
    - rag
    - campus
    - tvl-book
units:
  time: [milliseconds, seconds]
  currency: [USD]
  ratio: [dimensionless]
configuration_space:
  model:
    type: categorical
    values: ["gpt-4o-mini", "gpt-4o", "claude-3-haiku"]
  temperature:
    type: continuous
    range:
      min: 0.1
      max: 0.8
    unit: ratio
  max_tokens:
```

```yaml
    type: discrete
    values: [256, 512, 768]
    unit: tokens
  retrieval_depth:
    type: integer
    range:
      min: 2
      max: 6
    unit: count
objectives:
  - name: answer_quality
    direction: maximize
  - name: response_latency
    direction: minimize
  - name: token_cost
    direction: minimize
optimization:
  strategy: pareto_optimal
  pareto_config:
    selection_method: crowding_distance
  budget:
    max_trials:
      value: 60
      unit: count
    parallel_trials: 6
    timeout_seconds: 2400
constraints:
  - id: campus-hour-latency
    type: expression
    rule: 'params.response_latency_ms <= 1200'
    error_message: "Keep responses under 1.2s during campus rush"
  - id: premium-model-budget
    type: conditional
    when: 'params.model == "gpt-4o"'
    then: 'params.token_cost_usd <= 0.04'
    error_message: "Premium models need budget guardrails"
environments:
  base:
    objectives:
      - name: response_latency
        threshold:
          value: 900
          unit: milliseconds
  finals_week:
    extends: base
    overrides:
      configuration_space:
        retrieval_depth:
          range:
```

```yaml
            min: 3
            max: 8
      optimization:
        budget:
          max_trials:
            value: 90
            unit: count
evaluation:
  workloads:
    - name: onboarding_faq
      dataset: dvl://faq/onboarding
    - name: policy_changes
      dataset: dvl://faq/policy-2024
  metrics:
    answer_quality:
      aggregation: ndcg@10
    response_latency:
      aggregation: p95
    token_cost:
      aggregation: mean
promotion:
  gate: strict_dominance
  reviewers: ["ops@traigent.ai"]
triagent:
  pipeline: RAGOrientation
  deployment_targets: ["staging", "production"]
  manifest_output: artifacts/params.yaml
dvl:
  version: ">=0.4.0"
  drift_tests:
    - name: faq-topic-shift
      rule: 'metrics.topic_overlap >= 0.85'
```

1.5 How to Use This Book

Each chapter mixes narrative, tables, and executable TVL listings stored under `tvl/tvl_book/examples`. Read them in order for a guided tour, or jump straight to the integration chapter if you already know the language and care about Triagent and DVL. Try editing the listings and running your own validation scripts—TVL is happiest when it powers real experiments.

1.6 Field Notes Icons

Keep an eye out for three recurring call-outs:

- **Hackathon Moves** highlight shortcuts for rapid prototyping.
- **Reliability Pitfalls** warn you when a choice carries hidden operational risk.

- **Integration Hints** show how TVL interacts with Triagent and DVL in practice.

They are sprinkled through later chapters to keep the pacing fun while grounding every concept in how we run real systems at TraiGent Labs.

Chapter 2

Getting Fluent in TVL

2.1 First Contact

Pick up a TVL file and you should be able to answer three questions: what are we tuning, what does success look like, and who is accountable. Those answers live in different sections of the spec, and together they form a contract between engineers, operators, and the optimizer.

Section	Responsibilities
`spec`	Names the experiment and version so it can be tracked in git, DVC, and dashboards.
`metadata`	Declares owners, intent, and any review policy.
`configuration_space`	Declares the levers TVO can pull, along with their types and ranges.
`objectives`	Describes how we measure success and the direction we care about.
`optimization`	Configures the search strategy and budgets.
`constraints`	Encodes safety or business rules TVO must respect.
`evaluation`	Binds workloads and metrics so automated tests remain grounded.
`promotion`	Governs the path from candidate to production.

Table 2.1: TVL sections capture different responsibilities in the tuning lifecycle.

2.2 A Minimal Working Spec

The quickstart listing below is intentionally small. Read it top to bottom: each section builds on the previous one, painted with YAML but backed by TVL's schema validations.

Listing 2.1: Minimal TVL specification from Chapter 1.1's simpler cousin.

```yaml
spec:
  id: hello-tvl
  version: 0.0.1
metadata:
```

```
  owner: "students@traigent.ai"
  description: "Minimal TVL spec used in Chapter 2 to introduce the core
      sections."
  reviewers: ["mentor@traigent.ai"]
units:
  time: [milliseconds, seconds]
  ratio: [dimensionless]
configuration_space:
  model:
    type: categorical
    values: ["gpt-4o-mini", "claude-3-haiku"]
  temperature:
    type: continuous
    range: [0.0, 0.8]
    unit: ratio
  max_tokens:
    type: discrete
    values: [256, 384, 512]
    unit: tokens
objectives:
  - name: faithfulness
    direction: maximize
  - name: latency
    direction: minimize
optimization:
  strategy: pareto_optimal
  budget:
    max_trials:
      value: 30
      unit: count
    timeout_seconds: 900
constraints:
  - id: mild-temperature
    type: expression
    rule: 'params.temperature <= 0.8'
    error_message: "Stick to milder sampling in the quickstart."
evaluation:
  workloads:
    - name: tutorial
      dataset: dvl://demo/tutorial
promotion:
  gate: manual_review
  reviewers: ["mentor@traigent.ai"]
```

2.3 Line-by-Line Walkthrough

Start at the `spec` block and notice how the identifier, semantic version, and metadata form a paper trail. Down in `configuration_space`, we annotate the units so downstream services can display or

alert in the right scale. Each objective names the direction—`minimize` versus `maximize`—so you never wonder if lower latency is better (it usually is). Budgets give TVO the guardrails it needs to make smart choices without melting your GPU quota. Finally, promotion tells the human story: who reads the report before a change lands in production.

2.4 Typing Your Variables

TVL offers crisp typing for tuning variables:

- **Categorical** for discrete choices like model families.
- **Continuous** for floating-point ranges, optionally annotated with units.
- **Discrete** for enumerated integers that are meaningful as individual steps.
- **Integer** for ranged integers where the optimizer can choose any value between bounds.

This design keeps optimizers honest. For example, NSGA-II cannot accidentally propose half a retrieval depth, and gradient-based heuristics know which variables are safe to nudge smoothly.

2.5 Validation Warm-Up

Before we hand a spec to TVO we validate it locally. Chapter 2.1 ships with a tiny Python script that loads the YAML, checks for the fields we expect, and prints out the configuration space in a human-friendly table. The script lives beside the spec so you can extend it as you go.

Listing 2.2: Python helper that prints the configuration space and flags missing sections.

```python
"""Quick helper for Chapter 2 to inspect a TVL spec and print the knobs.
    """
from pathlib import Path
import textwrap

import yaml


def load_spec(path: Path) -> dict:
    with path.open("r", encoding="utf-8") as handle:
        return yaml.safe_load(handle)


def ensure_sections(spec: dict, required: list[str]) -> list[str]:
    missing = [section for section in required if section not in spec]
    return missing


def print_configuration_space(space: dict) -> None:
    print("Configuration Space")
    print("-------------------")
```

```
    for name, definition in space.items():
        type_ = definition.get("type", "unknown")
        unit = definition.get("unit")
        details = []
        if "values" in definition:
            details.append(f"values={definition['values']}")
        if "range" in definition:
            details.append(f"range={definition['range']}")
        info = ", ".join(details)
        if unit:
            info = f"{info} ({unit})" if info else f"unit={unit}"
        print(f"- {name} [{type_}]: {info}")


def main() -> None:
    spec_path = Path(__file__).with_name("ch2_hello_tvl.tvl.yml")
    spec = load_spec(spec_path)

    required_sections = ["spec", "metadata", "configuration_space", "
        objectives", "optimization"]
    missing = ensure_sections(spec, required_sections)
    if missing:
        print("Missing sections:", ", ".join(missing))
        return

    print(textwrap.dedent(
        """\
            Spec loaded successfully.
        id        : {id}
        version   : {version}
        owner     : {owner}
        """
    ).format(
        id=spec["spec"].get("id", "<unknown>"),
        version=spec["spec"].get("version", "<unknown>"),
        owner=spec["metadata"].get("owner", "<unknown>"),
    ))

    print_configuration_space(spec["configuration_space"])


if __name__ == "__main__":
    main()
```

2.6 Common Pitfalls

- Forgetting units on continuous variables (a quick way to ship a 2-second timeout instead of 2 ms).

- Mixing integers and strings in a categorical list, which fails validation but wastes time.
- Leaving objectives unspecified, forcing TVO to guess what matters.
- Setting generous budgets with no promotion gate—the optimizer may explore forever without oversight.

Treat these as checklists before you commit a spec.

2.7 Objectives and Budgets

TVO defaults to multi-objective optimization, so list every metric you care about. If you only want a single target (say latency), keep the list to one entry. Budgets live alongside objectives to make the trade-off explicit: we spend computation in exchange for better configurations.

2.8 Promotion as a First-Class Citizen

Even a university hackathon deserves decent governance. A promotion gate such as `manual_review` or `strict_dominance` ensures there is a human or automated check before experiments impact users. Couple this with CI so merges fail if the gate denies the promotion. The TVL spec tells tools what to enforce.

2.9 Interactive Lab: Orientation RAG Circuit

The web companion to this book hosts a microsimulation that mirrors the quickstart spec. Visit the Orientation RAG lab (`../sims/orientation-rag-circuit/index.html`) to adjust the sliders and watch latency, cost, and quality gauges react in real time. Each slider writes back to the same fields taught in this chapter, while constraint badges map to the CEL rules introduced in Chapter 3.1.

Offline readers. Scan the QR code in Appendix **??** or open the bundled offline guide to follow along when you cannot load the web version.

Chapter 3

Constraints, Units, and Safety Nets

3.1 Why Units Matter

One of the sharpest critiques from the TVL unification review was that ambiguous fields invite human error. When a teammate reads `threshold: 2000` they should not have to wonder whether it is milliseconds or seconds. TVL therefore asks you to declare canonical unit domains and annotate variables accordingly. During export, tools like `params.yaml` append the unit to the key name (`cache_ttl_hours`) so downstream code is unambiguous.

3.2 CEL in Plain English

Constraints in TVL use the *Common Expression Language* (CEL). It looks a little like Python expressions, but it is parsed and type-checked before each evaluation:

- **Expression constraints** evaluate to `true` or `false`.
- **Conditional constraints** evaluate a guard (`when`) and then enforce a rule.
- **Forbidden constraints** spell out anti-patterns you want to block outright.

CEL covers arithmetic, logic, membership, and function calls (string matching, list membership, math). Any constraint that returns something other than a boolean fails validation, making mistakes obvious.

3.3 Worked Example

The listing below demonstrates units, expressions, and conditional logic in one place. Open the file and try tweaking the numbers: what happens if you drop `cache_ttl_hours` below 1? How about pushing `rerank_weight` past 0.8? TVO will reject candidates that violate these rules before wasting a single trial.

Listing 3.1: Experiment exercising CEL constraints and explicit units.

```
spec:
  id: constraints-lab
```

```yaml
  version: 0.1.0
metadata:
  owner: "qa@traigent.ai"
  description: "Explores CEL constraints and unit typing."
units:
  time: [milliseconds, seconds]
  currency: [USD]
  ratio: [dimensionless]
  percentage: [percent]
configuration_space:
  temperature:
    type: continuous
    range: [0.0, 1.0]
    unit: ratio
  max_tokens:
    type: integer
    range: [128, 1024]
    unit: tokens
  cache_ttl_hours:
    type: continuous
    range: [0.5, 12.0]
    unit: hours
  rerank_weight:
    type: continuous
    range: [0.0, 1.0]
    unit: ratio
objectives:
  - name: stability
    direction: maximize
  - name: cost
    direction: minimize
constraints:
  - id: cache-ttl-unit-check
    type: expression
    rule: 'params.cache_ttl_hours >= 1.0'
    error_message: "Cache needs at least 1 hour TTL."
  - id: token-temperature-budget
    type: expression
    rule: 'params.temperature * params.max_tokens <= 600'
    error_message: "Token-temperature product keeps hallucinations
       manageable."
  - id: rerank-mandate
    type: conditional
    when: 'params.max_tokens >= 512'
    then: 'params.rerank_weight >= 0.4'
    error_message: "Long answers must lean on reranking."
  - id: rerank-cap
    type: forbidden
    rule: 'params.rerank_weight > 0.8'
```

```
    error_message: "Rerank weight above 0.8 is unstable."
evaluation:
  workloads:
    - name: stability_suite
      dataset: dvl://qa/stability-suite
  metrics:
    stability:
      aggregation: mean
    cost:
      aggregation: mean
```

3.4 Constraint Playbook

- Use **expression** constraints for universal truths (budget, latency, entropy caps).
- Reach for **conditional** constraints when a rule applies only to part of the space, such as premium models or weekend workloads.
- Deploy **forbidden** constraints to codify "never again" lessons learned from incidents.

Compose them freely—CEL is designed to be readable on pager duty at 3am.

3.5 Interactive Drill: Latency Spike Preset

Load the Orientation RAG circuit lab's *Latency Spike* preset to watch CEL rules fire as you drag sliders. The latency meter mirrors the constraint

```
params.latency_ms <= 800
```

and the conditional rerank guard highlights the component whenever `retriever_top_k >= 40` while `rerank_weight < 0.3`. Use the exercise to build intuition before encoding the same constraints in YAML.

3.6 Making Constraints Testable

Because CEL rules are declarative, you can unit-test them just like code. A simple pattern is to load the spec in Python, feed sample parameter dictionaries, and assert that the CEL engine returns `true`. This keeps the rules alive as the configuration space evolves and guards against regressions when teammates add overrides.

Listing 3.2: Pytest-style checks that validate CEL expressions against sample assignments.

```python
"""Example pytest module that exercises the Chapter 3 CEL constraints."""
from __future__ import annotations

from pathlib import Path
from types import SimpleNamespace
```

```python
from typing import Any, Dict
import re

import yaml


SPEC_PATH = Path(__file__).with_name("ch3_constraints_units.tvl.yml")
SPEC = yaml.safe_load(SPEC_PATH.read_text(encoding="utf-8"))


def evaluate_constraint(expr: str, params: Dict[str, Any]) -> bool:
    """Tiny CEL-like evaluator for demonstration purposes."""
    python_expr = expr.replace("&&", " and ").replace("||", " or ")
    python_expr = re.sub(r"!(?!=)", " not ", python_expr)
    activation = {"params": SimpleNamespace(**params)}
    return bool(eval(python_expr, {}, activation))


def test_temperature_token_budget_passes() -> None:
    params = {
        "temperature": 0.5,
        "max_tokens": 512,
        "cache_ttl_hours": 2.0,
        "rerank_weight": 0.5,
    }
    expr = next(c["rule"] for c in SPEC["constraints"] if c["id"] == "
       token-temperature-budget")
    assert evaluate_constraint(expr, params)


def test_temperature_token_budget_fails() -> None:
    params = {
        "temperature": 0.95,
        "max_tokens": 768,
        "cache_ttl_hours": 3.0,
        "rerank_weight": 0.6,
    }
    expr = next(c["rule"] for c in SPEC["constraints"] if c["id"] == "
       token-temperature-budget")
    assert not evaluate_constraint(expr, params)


def test_rerank_requirement_triggers() -> None:
    constraint = next(c for c in SPEC["constraints"] if c["id"] == "rerank
       -mandate")
    params = {
        "temperature": 0.3,
        "max_tokens": 640,
        "cache_ttl_hours": 1.5,
```

```
        "rerank_weight": 0.2,
    }
    when_ok = evaluate_constraint(constraint["when"], params)
    then_ok = evaluate_constraint(constraint["then"], params)
    assert when_ok and not then_ok
```

3.7 Debugging Failures Fast

When TVO rejects a candidate it returns the failing constraint identifier. Pair that identifier with the `error_message` you wrote and log the parameter set. Most issues fall into one of three buckets:

- The override expanded a range without updating the constraint (narrow it or update the rule).
- A unit conversion went missing (milliseconds vs. seconds).
- The dataset changed shape and a derived metric (like `topic_overlap`) now has a different scale—time to revisit the DVL suite.

Capture the fix in Git history so future debuggers see the reasoning.

Chapter 4

Patterns for Real Deployments

4.1 Inheritance Without Surprises

TVL's `extends` and `overrides` mechanics turn sprawling configuration fleets into layered, auditable structures. The rules are simple but strict: you may narrow ranges, remove choices by taking subsets, and adjust thresholds, but you cannot change types or add new categorical values. This mirrors the validated design from the TVL unification document and prevents environment-specific overrides from drifting into a different schema altogether.

4.2 Staging vs. Production, Codified

The listing below captures a common dance: staging accepts wider experimentation, production tightens budgets and latency goals. Both inherit from the same base so operators can reason about differences line-by-line. Try diffing the two overlays to see how each environment tailors the knobs.

Listing 4.1: Environment overlays tighten ranges and thresholds without breaking type safety.

```
spec:
  id: overlays-fleet
  version: 0.2.0
metadata:
  owner: "platform@traigent.ai"
  description: "Shows how extends/overrides manage environments."
units:
  time: [milliseconds, seconds]
  ratio: [dimensionless]
  currency: [USD]
configuration_space:
  model:
    type: categorical
    values: ["gpt-4o-mini", "claude-3-sonnet", "claude-3-haiku"]
  temperature:
    type: continuous
    range: [0.2, 0.9]
    unit: ratio
```

```yaml
  max_tokens:
    type: integer
    range: [128, 1024]
    unit: tokens
  reranker:
    type: categorical
    values: ["e5-large", "voyage-2", "semantic-router"]
objectives:
  - name: helpfulness
    direction: maximize
  - name: latency
    direction: minimize
  - name: cost
    direction: minimize
optimization:
  strategy: pareto_optimal
  budget:
    max_trials:
      value: 80
      unit: count
    parallel_trials: 8
    timeout_seconds: 3600
environments:
  staging:
    overrides:
      configuration_space:
        model:
          values: ["gpt-4o-mini", "claude-3-haiku"]
        temperature:
          range: [0.2, 0.6]
      optimization:
        budget:
          max_trials:
            value: 30
            unit: count
  production:
    extends: staging
    overrides:
      configuration_space:
        max_tokens:
          range: [256, 768]
      objectives:
        - name: latency
          threshold:
            value: 750
            unit: milliseconds
      promotion:
        gate: strict_dominance
constraints:
```

```
  - id: premium-model-reranker
    type: conditional
    when: 'params.model == "claude-3-sonnet"'
    then: 'params.reranker in ["voyage-2", "semantic-router"]'
    error_message: "Premium models require smarter reranking."
triagent:
  pipeline: SupportChat
  deployment_targets: ["staging", "production"]
```

4.3 Constraint-Aware Overlays

Inside overrides you can encode stricter constraints for premium pathways or high-risk deployments. In the example, production insists on a 750 ms latency ceiling and restricts the long-form `max_tokens`. Because TVL validates the entire extended spec, these rules compose safely with base-level constraints.

4.4 When to Extend vs. Duplicate

If a new workload shares 80% of its DNA with an existing spec, extend it. You inherit guardrails, unit domains, and metadata automatically. Reach for a fresh spec only when the configuration space is wildly different or the objectives no longer align. This discipline keeps your repository small and consistent, much like a well-factored codebase.

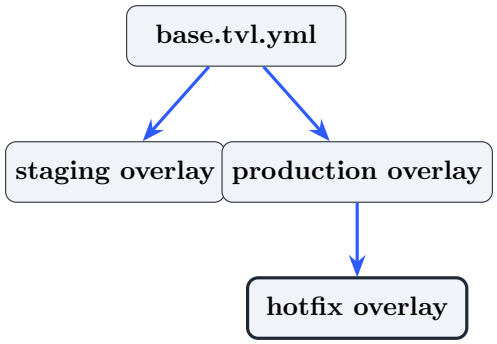

Figure 4.1: Layer overlays instead of copying specs. Hotfixes hang off production and are easy to remove later.

4.5 Optimization Playbooks

Multi-objective strategies like Pareto selection reward well-specified objectives. If you need a scalar score for ranking, consider the `display_weights` pattern from the integration design: keep weights only for presentation, not for the optimizer itself. This encourages healthy disagreement between objectives and surfaces trade-offs to reviewers instead of hiding them in a magic number.

4.6 Hotfix Overlays

Sometimes production needs a temporary constraint—a vendor outage, a compliance request, a surprise price hike. Instead of editing the base spec, create a short-lived overlay that narrows the affected ranges and annotate it with an expiration. Once the incident passes, delete the overlay and history will show exactly what changed.

Listing 4.2: Production hotfix overlay that temporarily caps premium model usage.

```yaml
extends: overlays-fleet
metadata:
  owner: "sre@traigent.ai"
  description: "Temporary cap to avoid premium model saturation during
     vendor outage."
  expires_after: "2025-01-15"
overrides:
  configuration_space:
    model:
      values: ["gpt-4o-mini", "claude-3-haiku"]
    temperature:
      range: [0.2, 0.5]
  optimization:
    budget:
      max_trials:
        value: 25
        unit: count
  constraints:
    - id: temporary-cost-cap
      type: expression
      rule: 'params.cost_usd <= 0.025'
      error_message: "Hold cost under temporary ceiling during outage."
```

4.7 Provenance and Promotion

Real pipelines lean on deterministic exports (e.g. `params.yaml`, `tvl.lock`, `provenance.json`). Make sure your CI pipeline checks these files in alongside the TVL spec. When the promotion gate enforces strict dominance, TVO must prove that the new candidate beats the incumbent on every objective without bending constraints. The paperwork may feel formal, but it pays dividends when you audit a deployment months later.

Chapter 5

TVL Meets Triagent and DVL

5.1 Unifying the Stack

The refined integration plan positions TVL as the single source of truth, TVO (within Triagent) as the governed optimizer, and DVL as the data validation guardrail. The flow is deterministic: TVL defines the configuration space, TVO evaluates candidates and produces promotion proofs, and DVL verifies that the data feeding those experiments remains trustworthy.

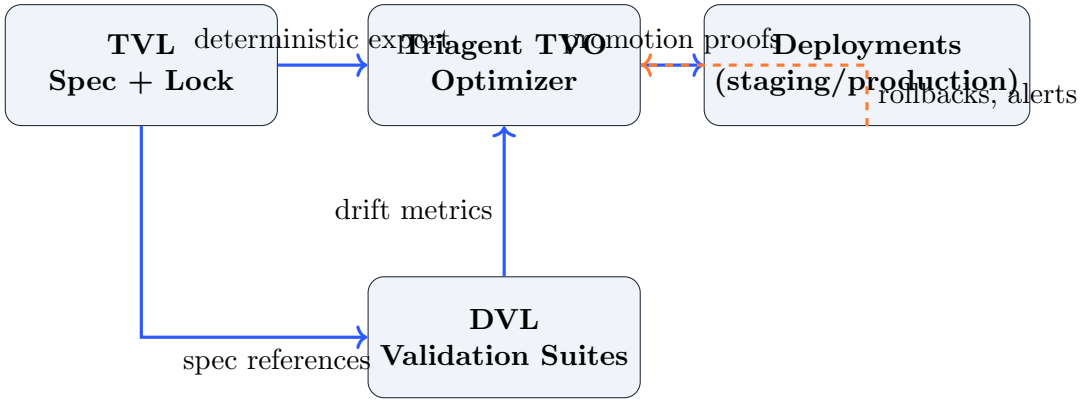

Figure 5.1: TVL anchors the configuration graph, Triagent/TVO executes governed optimization, and DVL feeds drift-aware validation signals.

5.2 Deterministic Artifacts

The integration spec insists that `params.yaml` remain deterministic, free of timestamps or noisy metadata. Instead, volatile fields move to `provenance.json`. This separation gives us reproducible runs, clean diffs, and clear audits. Always commit `tvl.lock`, `params.yaml`, and `provenance.json` together so reviewers see both the declarative intent and the resolved values.

5.3 Handing Off to Triagent

TVO reads the TVL spec directly, resolves overrides, and schedules evaluations. When a candidate dominates the incumbent on every objective (respecting constraints) the promotion gate flips. For staging environments, we often prefer manual review; production leans on `strict_dominance`. Triagent keeps the audit trail: who triggered the run, which seeds were used, and what rollback path is configured.

5.4 Lifecycle of a Promotion

1. Author updates the TVL spec and pushes it through code review.
2. CI calls `tvl validate` and `tvl export` to generate deterministic artifacts.
3. Triagent launches TVO with the resolved spec and recorded workloads.
4. DVL validates datasets before and after the run, surfacing drift signals.
5. Promotion gate checks strict dominance and reviewer sign-off.
6. Deployment manifests (like Listing 5.1) capture the approved outputs.

Each step leaves a breadcrumb trail—logs, manifests, and audit records—so you can reconstruct decisions weeks later.

5.5 Interactive Scenarios

The Orientation RAG circuit lab ships with presets for the three incidents referenced here: baseline, latency spike, and budget relief. Replay them at `../sims/orientation-rag-circuit/index.html` to see how latency, cost, and quality trade-offs influence promotion decisions. The `Copy YAML` button mirrors the `params.yaml` export that feeds Triagent.

5.6 DVL Keeps the Data Honest

TVL captures the configuration; DVL certifies the datasets. Before promoting a configuration, Triagent invokes DVL suites to check for drift, PII leaks, or other regressions. Failed DVL checks block promotion even if the metrics look stellar. Treat DVL as equal peers in the pipeline: configuration without clean data is misleading, and clean data without tuned parameters under-delivers.

5.7 A Manifest That Glues It Together

Here is a manifest that a CI job could produce after a successful TVO run. It references the TVL spec, records the resolved SHA, and lists the validation jobs that must pass before deployment.

Listing 5.1: Deployment manifest binding TVL, Triagent, and DVL checks together.

```
# Generated deployment manifest linking TVL, Triagent (TVO), and DVL.
manifest:
  spec_path: tvl/spec/rag-campus-orientation.tvl.yml
  spec_sha: 7c2f31be
  tvl_lock: tvl.lock
  provenance: provenance.json
triagent:
  pipeline: RAGOrientation
  optimizer: TVO
  deployment_target: production
  evaluation_job: dvc repro deploy-rag-orientation
  promotion_gate: strict_dominance
  rollback_on:
    - latency_regression
    - constraint_violation
dvl:
  dataset_version: faq/v2024.10
  drift_tests:
    - faq-topic-shift
    - forbidden-pii
  validation_suite: dvl suites/faq-orientation.json
integration_checks:
  - tvl validate tvl/spec/rag-campus-orientation.tvl.yml
  - tvl export --lock tvl/spec/rag-campus-orientation.tvl.yml
  - dvl validate suites/faq-orientation.json
  - triagent deploy --dry-run RAGOrientation
```

5.8 Automation Recipe

The shell pipeline below mirrors what we run in CI: validate the spec, execute a dry run, run DVL suites, and collect the manifest. Use it as a template and extend it with notifications or rollback hooks.

Listing 5.2: Shell pipeline orchestrating TVL validation, Triagent dry run, and DVL checks.

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SPEC="${ROOT_DIR}/ch1_motivation_experiment.tvl.yml"
MANIFEST="${ROOT_DIR}/ch5_integration_manifest.yaml"

echo "[validate] Checking spec..."
tvl validate "${SPEC}"

echo "[export] Producing deterministic artifacts..."
tvl export --lock "${SPEC}"
```

```
echo "[triagent] Dry-running pipeline..."
triagent deploy --dry-run RAGOrientation

echo "[dvl] Running validation suites..."
dvl validate suites/faq-orientation.json

echo "[manifest] Capturing manifest..."
cat "${MANIFEST}"
```

5.9 TVL Tooling and Plug-ins

Behind the `tvl` command is a constellation of plug-ins housed in `tvl_tools/`. Each module wraps a stage of structural or operational validation so you can mix them in CI or local workflows.

Module	Purpose
`tvl_parse`	Fast parser that ensures specs conform to the schema before any expensive checks.
`tvl_validate`	End-to-end validation including CEL evaluation, units, and override semantics.
`tvl_check_structural`	Lints inheritance graphs, duplicate IDs, and naming conventions.
`tvl_check_operational`	Evaluates runtime guardrails such as budget sizing and environment overlays.
`tvl_lint`	Style fixer for YAML layout, ordering, and comment hygiene.
`tvl_ci_gate`	Aggregates structural and operational checks for promotion pipelines.
`tvl_measure_validate`	Validates metric definitions and unit annotations referenced in specs.
`tvl_config_validate`	Validates CLI overrides and environment-specific diffs before deployment.

Table 5.1: Key plug-ins exposed through the `tvl` CLI. Pick and combine them in CI pipelines.

Most teams call these modules directly from Python or wrap them in `nox`/`tox` sessions. When Triagent assembles a promotion proof, it invokes `tvl_ci_gate`, collects reports, and attaches them to the manifest so reviewers see exactly which guardrails passed.

5.10 Looking Ahead

Future integrations will automate even more of this loop: Triagent can stream live metrics back into TVL, update `tvl.lock` with derived constraints, and share DVL drift annotations so engineers spot which workloads deserve new objectives. Keep specs modular, keep units explicit, and let the tooling shoulder the boring bits so you can focus on designing better experiences.

Appendix A

Quick Reference and Resources

A.1 Interactive Lab Access

The TVL Intelligent Textbook ships with an Orientation RAG microsimulation that accompanies the walkthroughs in Chapters 2–5. When reading the web edition the lab is embedded inline; offline readers can launch it locally by opening `intelligent-textbooks/docs/sims/orientation-rag-circuit/index.` or by serving the site with `mkdocs serve` from `tvl/tvl_book/intelligent-textbooks`.

To regenerate scenario presets after editing a spec, run:

```
python -m tvl_tools.microsim_bridge path/to/spec.tvl.yml \
  --output tvl/tvl_book/intelligent-textbooks/docs/sims/orientation-rag-circuit/microsim_prese
```

This command snaps slider ranges to the configuration space, emits baseline plus incident presets, and records the spec identifier for provenance. Rebuild the MkDocs site or refresh the standalone HTML page to see the updates.

A.2 Compiling This Book

- Install a modern LaTeX distribution (TeX Live 2022+ recommended).
- From `tvl/tvl_book`, run `pdflatex main.tex` twice for stable references.
- Optional: add `biber` or `biblatex` if you extend the book with citations.

A.3 Essential Commands

- `tvl validate <spec>` — schema and CEL validation.
- `tvl export <spec> --lock` — produce deterministic `params.yaml` and `tvl.lock`.
- `triagent run <pipeline>` — execute TVO-driven optimization jobs.
- `dvl validate <suite>` — run dataset quality gates referenced in the spec.

A.4 Further Reading

- SANER Extended Abstract: motivation for governed tuned variables.
- TVL Unification Design v2.0: architecture decisions and deterministic export rules.
- DVL Drift Playbook: recommended validation suites for common workloads.

A.5 Exploring the Examples

- Chapter 1–2 specs live under `examples/ch1_*.tvl.yml` and `examples/ch2_*.tvl.yml`. Swap variable ranges and rerun the validation script to see error messages.
- Chapter 3 adds `ch3_constraint_tests.py`; run it with `pytest` to watch CEL rules fire.
- Chapter 4's overlays include `ch4_hotfix_overlay.tvl.yml`—practice removing it once a simulated incident ends.
- Chapter 5 bundles `ch5_integration_pipeline.sh`; adapt it to your CI runner.

A.6 Toolchain Map

The `tvl` CLI is modular. Each plug-in under `tvl_tools` can be invoked directly or composed in automation scripts. Use this cheat sheet when wiring them into CI.

Command	**What it Checks**
`python -m tvl_tools.tvl_parse <spec>`	Schema validation and fast parse reporting line-level issues.
`python -m tvl_tools.tvl_validate <spec>`	Full semantic validation, CEL constraints, and units.
`python -m tvl_tools.tvl_check_structural <spec>`	Inheritance graph linting, duplicate detection, naming.
`python -m tvl_tools.tvl_check_operational <spec>`	Budget sanity, environment overlays, deployment readiness.
`python -m tvl_tools.tvl_lint <spec>`	Formatting, canonical key ordering, and comment cleanup.
`python -m tvl_tools.tvl_ci_gate <spec>`	Bundled gate used by Triagent promotion pipelines.
`python -m tvl_tools.tvl_measure_validate <metrics.yml>`	Ensures metric definitions and units match expectations.
`python -m tvl_tools.tvl_config_validate --override <file>`	Validates environment overrides or CLI patch files.
`python -m tvl_tools.microsim_bridge <spec>`	Generates Orientation RAG MicroSim presets for the interactive lab.

Table A.1: TVL plug-ins and the checks they perform. Combine them to match your deployment policies.

Acknowledgments

This book draws on the TraiGent TVL specification, the SANER extended abstract, and the TVL–TVO integration design. We thank the engineers, researchers, and reviewers who shaped the ecosystem.